信盈达 • C 语言期末冲刺

易错题与核心知识点深度解析

按模块分类 • 「错误表现 × 正确逻辑 / 代码」对照 • 6 大模块 17 个考点

怎么用这份文档：每张卡片左红右绿——左边是考场上最容易踩的坑，右边是标准结论或正确代码，下面才是原理。复习时先遮住右侧自己答一遍，再看解析。标签标出该考点在你两份卷子里的表现，「两卷连错」的两张卡建议优先背。

模块一 数据类型与内存大小

模块二 指针与数组（重灾区）

模块三 函数与参数传递

模块四 运算符与宏定义

模块五 流程控制与文件操作

模块六 编程算法逻辑

标签说明：两卷连错　卷一错题　已补上　补充考点 / 已掌握

模块一　数据类型与内存大小

3 个考点

1.1 结构体内存对齐（高频大题 / 选择题） 已补上

错误表现

计算 `struct Node { char a; int b; short c; };` 时，算出 8 或 10 字节（漏算中间填充或漏算尾部补齐）。

正确结果

64 位系统默认对齐下，该结构体大小为 12 字节。

三步走口诀：找最大 → 排座位 → 算总长

1. 找对齐数：系统默认 8，最大成员 `int` 为 4，取较小者 → 对齐数 4。
2. 排座位（看偏移量）：`char a` 偏移 0，占 1 字节（位置 0）；`int b` 自身 4，偏移必须是 4 的倍数，位置 1 不是 → 补 3 个空字节，b 放在 4~7；`short c` 自身 2，位置 8 是 2 的倍数 → 放在 8~9。
3. 算总长：当前 10 字节，必须是最大对齐数 4 的整数倍 → 补齐到 12 字节。

对照练习：`struct A { char a; int b; };` → 1 + 补3 + 4 = 8；`struct S { int a; char b; };` → 4 + 1 + 补3 = 8。

1.2 数据溢出与回环 补充考点

错误表现

认为 `char a = 128; printf("%d", a);` 输出 128。

正确结果

输出 -128。想输出 128，必须写 `unsigned char`。

原理：常见平台上 `char` 是有符号的，范围 -128~127。128 的二进制是 `1000 0000`，最高位为 1 代表负数，按补码规则它表示的就是 -128，超出范围后从最小端回环。

严谨提示：`char` 是否有符号由实现定义；GCC / MSVC 的 x86 默认 signed，上述结论成立。写跨平台代码时请显式用 `signed char` / `unsigned char`。

1.3 sizeof 与 strlen 的区别 已掌握

错误表现

把 `char str[] = "hello";` 的 `sizeof` 和 `strlen` 都算成 5。

正确结果

`sizeof(str)` = 6（含末尾 `'\0'`）；`strlen(str)` = 5（遇 `'\0'` 停止，不计入）。

一句话：`sizeof` 是运算符，算的是「占多少内存」；`strlen` 是库函数，算的是「到 `'\0'` 之前有几个字符」。若写成 `char *p = "hello";`，`sizeof(p)` 就变成指针大小 8，与字符串长度无关。

模块二　指针与数组（重灾区）

3 个考点

2.1 字符串常量区不可修改 已掌握

错误表现

认为 `char *p = "hello"; p[1] = 'E';` 可以正常运行。

正确结果

程序崩溃（段错误）。只有 `char buf[] = "hello";` 才能改。

原理：`char *p = "hello";` 中的 `"hello"` 存放在只读常量区，p 只是指向它，绝对不允许写入；`char buf[] = "hello";` 是在栈区申请了一份可写副本。规范写法是 `const char *p = "hello";`，让编译器提前拦住误写。

对应原卷找错/选择题：`p[0] = 'H';` 是问题行，`a[0] = 'W';` 合法。

2.2 数组传参的退化陷阱 已掌握（概念）

错误表现

形参写 `int arr[]`，却在函数内部用 `sizeof(arr) / sizeof(arr[0])` 求数组长度。

正确结果

`sizeof(arr)` 得到的是指针大小 8 字节（64 位），该表达式恒为 2（8/4），与真实长度无关。必须额外传 `len` 参数。

原理：数组作为函数参数传递时会自动退化为指针，函数内部拿到的只是一个地址，并不知道数组有多长。`int arr[]` 与 `int *arr` 作形参完全等价。

正确写法：`void f(int arr[], int len) { for (int i = 0; i < len; i++) ... }`

已校正：原稿写「结果是指针大小（64 位下 8 字节）」，严格说是 `sizeof(arr)` 为 8，整个比值的结果是 2；这里按标准 C 表述更正，结论「永远算不对」不变。

2.3 指针偏移与二维数组 卷一错题

错误表现

把 `*(a + 1) + 2` 当作去取数组里的元素值。

正确结果

它是个地址。要取值必须再解一层：`*(*(a + 1) + 2)`，等价于 `a[1][2]`。

原理：`int a[3][4];` 中 `a + 1` 是「指向第 1 行（4 个 int 的数组）」的指针；`*(a + 1)` 得到第 1 行这个数组，在表达式中退化为指向 `a[1][0]` 的 `int *`；再加 2 仍然只是地址 `&a[1][2]`，外面必须补 `*` 才拿到值。

另外两个等价写法：`*(a[1] + 2)`、`*(&a[0][0] + 6)`（按一维连续存储算偏移 1*4+2 = 6）。

同理：`int *p = &a[1];` 时 `p[2]` 是 `a[3]`，下标永远相对 p 的起点，不相对数组首元素。

模块三　函数与参数传递

2 个考点

3.1 指针传参的「形参陷阱」 基础

错误表现

`void getMemory(char *p) { p = malloc(10); }`，认为主函数能拿到分配的空间。

正确结果

主函数实参依然是 `NULL`，且这块内存再也找不到 → 内存泄漏。

原理：函数内的 `p` 只是实参的一份拷贝，`p = ...` 只改了局部变量的指向。要修改外部指针的指向，必须传递二级指针（或返回新地址）。

正确代码

```
void getMemory(char **p) {
    *p = malloc(10);
}
/* 调用：char *buf = NULL; getMemory(&buf); */
```

同型考点：`void my_strcpy(char *dest, char *src) { dest = src; }` 也是同一个错——改副本，一个字符都没拷贝。

3.2 结构体指针访问数组成员 卷一编程题错题

错误表现

传入 `struct Student *p`（实际指向结构体数组），函数里写 `scanf("%s", p->stu[i].name);`。

正确结果

编译报错：结构体里没有 `stu` 成员。应写 `p[i].name` 或 `(p + i)->name`。

原理：`p` 指向的是数组的第 0 个元素，`p + i` 才走到第 i 个元素。结构体里只有 `name`、`id`、`scores[3]`，没有名为 `stu` 的字段。

配套细节：`scanf("%d", &p[i].id)` 要取地址；`p[i].name` 是数组名，本身退化为地址，不用加 `&`；3 门课要用循环写 `p[i].scores[j]`。

模块四　运算符与宏定义

4 个考点

4.1 宏定义的「纯粹替换」 已掌握

错误表现

`#define M(a) a * a`，计算 `M(3+4)` 得出 49。

正确结果

19。宏不做计算，只做文本替换。

展开过程：`M(3+4)` → `3 + 4 * 3 + 4`，先算乘法 → `3 + 12 + 4 = 19`。

正确写法：参数与整体都要加括号 → `#define M(a) ((a) * (a))`。

4.2 宏定义与自增运算（未定义行为） 卷一错题

错误表现

`#define CUBE(x) (x * x * x)` 传入 `n++`，算出 24 且认为 n 变成 4。

正确结果

展开成 `(n++ * n++ * n++)`，属未定义行为：不同编译器结果不同，GCC 下通常是结果 24、n 为 5。考试遇到直接判错或避开。

原理：同一表达式里对 `n` 三次自增，中间没有序列点，标准不规定求值顺序。稳妥写法是先算好值再传：`int t = n; int r = CUBE(t); n++;`，且宏本身应写成 `((x)*(x)*(x))`。

4.3 复合赋值运算符 两卷连错 · 优先背

错误表现

`int a=10; a += a -= a * a;` 算出 0 或 -80。

正确结果

-180。

从右向左拆：

1. 先算内层 `a -= a * a`：`a * a = 100`，`a = 10 - 100 = -90`，此时 a 立刻变成 -90，整个子表达式的值也是 -90。
2. 再算外层 `a += (-90)`：此时 a 已是 -90，`-90 + (-90) = -180`。

同型题：`a *= 2 + 3` 等价于 `a = a * (2 + 3)`，不是 `a * 2 + 3` —— 复合赋值优先级低于算术运算。

严谨提示：按 C 标准，同一表达式内对 `a` 两次修改且无序列点属未定义行为；期末与笔试一律按教材结论 -180 记忆即可。

4.4 短路特性 已补上

错误表现

`int x=5, y=0; if (x++ || ++y)` 认为 y 会变成 1。

正确结果

y 依然是 0（x 变成 6）。

原理：`x++` 先取旧值 5（为真），`||` 左边为真已能决定结果，发生短路，右边 `++y` 根本不执行。
镜像题：`if (x++ && ++y)` 且 `x=0` 时，左值为 0（假），`++y` 同样被跳过，x 变 1、y 仍为 0。

答题术语写「短路 / 短路求值」，不要写「前置条件判断」这类自造说法。

模块五　流程控制与文件操作

2 个考点

5.1 循环中的 break 和 continue　补充考点

错误表现

`for(i=1;i<=5;i++) { if(i%2==0) continue; if(i==5) break; printf("%d ",i); }` 认为输出 5。

正确结果

输出 1 3。

逐步走：`i=1` 打印 1；`i=2, 4` 遇 `continue` 直接结束本次循环，不打印；`i=3` 打印 3；`i=5` 遇 `break` 整个循环终止，后续打印不执行。

区别：`continue` 跳过「本次剩余语句」，`break` 结束「整个循环」。

5.2 文件指针回到开头　两卷连错 · 优先背

错误表现

试图用 `fseek(fp, size, SEEK_CUR)` 回到文件开头；或把函数名写成 `fwind(fp)`。

正确结果 / 代码

直接写 `rewind(fp);` 或 `fseek(fp, 0, SEEK_SET);`。C 库里没有 fwind 这个函数。

三个定位常量：`SEEK_SET` 文件头、`SEEK_CUR` 当前位置、`SEEK_END` 文件尾。`SEEK_CUR` 是相对当前位置偏移，不可能一步回到开头。

顺带记模式：`"w+"` 清空原内容、指针在头；`"a+"` 保留原内容、可读可写、写位置在末尾；文件不存在时 `"w+"` / `"a+"` 都会新建，不会返回 NULL。

模块六　编程算法逻辑

3 个考点

6.1 找最值（打擂台法）　卷一编程题错题

错误表现

用类似冒泡的交换逻辑找最大值：`if (a[i] > a[i+1]) { a[i+1] = a[i]; }` —— 既会越界访问 `i+1`，又破坏了原数据，还根本没求总分。

正确逻辑

只用一个 `max_index` 记录下标，全程不改数组；每个学生先累加 3 门课得总分，再与当前最高分比较。

正确代码

```
int max_index = 0; // 假设第0个最大
for (int i = 1; i < n; i++) {
    if (stuArr[i].score > stuArr[max_index].score) {
        max_index = i; // 只记录下标，绝不更改原数据
    }
}
// 结束后打印 stuArr[max_index]
```

期末编程题要求的是「总分最高」，务必先用内层循环 `sum += stuArr[i].scores[j]` 累加，再打擂台；循环写 `i < n`，不要出现 `i+1`。

6.2 字符串 / 数组赋值　卷一、卷二找错题

错误表现

`char str[10]; str = "hello";` 或 `s[0].name = "张三";`；还有 `s[0].score = {90, 85, 88};`。

正确结果 / 代码

全部编译报错：数组名是常量，不能被整体赋值；初始化列表只能在定义变量时使用。用 `strcpy(str, "hello");` 或逐元素赋值 / 循环。

正确代码

```
#include <string.h>
strcpy(str, "hello");          /* 普通字符数组 */
strcpy(s[0].name, "张三");     /* 结构体里的字符数组 */
s[0].score[0] = 90;           /* 数组成员逐个赋值 */
s[0].score[1] = 85;
s[0].score[2] = 88;
```

顺带记：字符串拷贝必须连 `'\0'` 一起复制，标准写法 `while ((*dest++ = *src++) != '\0');`；用 `strcpy` 时要包含 `<string.h>` 并保证 dest 空间足够。

6.3 冒泡排序（结构体版） 补充考点

错误表现

排序时只交换分数，结果姓名和分数对不上号。

正确逻辑

比较用某个字段，交换时必须整个结构体一起换（连名带分一起搬）。

正确代码

```
for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - 1 - i; j++) {
        if (stuArr[j].score > stuArr[j+1].score) {
            struct Student temp = stuArr[j]; // 整个结构体一起换
            stuArr[j] = stuArr[j+1];
            stuArr[j+1] = temp;
        }
    }
}
```

两个易错点：内层上界是 `n - 1 - i`（防止访问 `j+1` 越界）；结构体可以直接整体赋值，`temp` 的类型必须写全 `struct Student`。

考前速查表

17 个考点一句话结论

模块	考点	一句话记住
一	结构体对齐	找最大 → 排座位 → 总长补到对齐数整数倍；`char/int/short` = 12
一	char 溢出	`char a=128` 输出 -128；要 128 用 `unsigned char`
一	sizeof / strlen	`sizeof` 含 `'\0'` = 6；`strlen` 不含 = 5
二	字符串常量	`char *p="..."` 只读不可改；`char a[]="..."` 可改
二	数组传参退化	函数内 `arr` 是指针，长度必须另传 `len`
二	二维数组解引用	`*(a+1)+2` 是地址；取值要 `*(*(a+1)+2)`
三	指针形参	改外部指针的指向，必须传二级指针 `**p`
三	结构体数组	`p[i].name` 或 `(p+i)->name`，没有 `p->stu[i]`
四	宏替换	`M(3+4)` = `3+4*3+4` = 19；写成 `((a)*(a))`
四	宏 + 自增	`CUBE(n++)` 未定义行为，考场判错；GCC 常见 24 / n=5
四	复合赋值	右结合、优先级最低；`a += a -= a*a` = -180
四	短路求值	左边能定结果，右边不执行；`||` 左真、`&&` 左假都短路
五	break / continue	`continue` 跳本次，`break` 结束整个循环；输出 1 3
五	文件复位	`rewind(fp)` / `fseek(fp,0,SEEK_SET)`；没有 fwind
六	打擂台	只记 `max_index`，不改原数组，不写 `i+1`
六	数组赋值	数组名是常量，用 `strcpy` 或逐元素赋值
六	结构体排序	比较按字段，交换整个结构体；内循环上界 `n-1-i`